

Gestione Segnali

Un **Segnale** è una notifica a un processo che è occorso un certo **Evento**.

Fra i tipi di Eventi che causano il fatto che il kernel generi un segnale per un processo ci sono i seguenti:

- **È occorsa una eccezione hardware:** L'HW ha verificato una condizione di errore che è stata notificata al kernel, il quale a propria volta ha inviato un segnale corrispondente al processo in questione.
 - Es: divisioni per 0, o riferimenti a parti di memoria inaccessibili.
- **L'utente ha digitato sul terminale dei caratteri speciali che generano i segnali.**
 - Es: il carattere interrupt (Control-C) e il suspend carattere (Control-Z)
- **È occorso un evento software.**
 - Es: un timer è arrivato a 0, il tempo di processore per il processo è stato superato, o un figlio del processo è terminato.

I Segnali sono definiti con interi unici, la cui sequenza inizia da 1.

Tali interi sono definiti in `<signal.h>` (o in `<sys/signal.h>`) con **Nomi Simbolici della forma SIGxxxx**.

Poiché gli effettivi numeri utilizzati per ogni segnale variano a seconda delle implementazioni, all'interno dei programmi è meglio utilizzare questi nomi.

Esempio: quando l'utente digita il carattere dell'interrupt, SIGINT (il segnale numero 2) è inviato a un processo.

No	Name	Default Action	Description
1	SIGHUP	terminate process	terminal line hangup
2	SIGINT	terminate process	interrupt program
3	SIGQUIT	create core image	quit program
4	SIGILL	create core image	illegal instruction
5	SIGTRAP	create core image	trace trap
6	SIGABRT	create core image	abort program (formerly SIGIOT)
7	SIGEMT	create core image	emulate instruction executed
8	SIGFPE	create core image	floating-point exception
9	SIGKILL	terminate process	kill program
10	SIGBUS	create core image	bus error
11	SIGSEGV	create core image	segmentation violation
12	SIGSYS	create core image	non-existent system call invoked
13	SIGPIPE	terminate process	write on a pipe with no reader
14	SIGALRM	terminate process	real-time timer expired
15	SIGTERM	terminate process	software termination signal
16	SIGURG	discard signal	urgent condition present on socket
17	SIGSTOP	stop process	stop (cannot be caught or ignored)
18	SIGTSTP	stop process	stop signal generated from keyboard
19	SIGCONT	discard signal	continue after stop
20	SIGCHLD	discard signal	child status has changed
21	SIGTTIN	stop process	background read attempted from control terminal
22	SIGTTOU	stop process	background write attempted to control terminal
23	SIGIO	discard signal	I/O is possible on a descriptor (see fcntl(2))
24	SIGXCPU	terminate process	cpu time limit exceeded (see setrlimit(2))
25	SIGXFSZ	terminate process	file size limit exceeded (see setrlimit(2))
26	SIGVTALRM	terminate process	virtual time alarm (see setitimer(2))
27	SIGPROF	terminate process	profiling timer alarm (see setitimer(2))
28	SIGWINCH	discard signal	Window size change
29	SIGINFO	discard signal	status request from keyboard
30	SIGUSR1	terminate process	User defined signal 1
31	SIGUSR2	terminate process	User defined signal 2

man 7 signal

Ciclo di Vita dei Segnali

- Si dice che **un segnale è generato da qualche Evento**.
- Dopo essere stato generato, **un segnale è inviato ad un processo**, che quindi **esegue una qualche azione in risposta al segnale**.
- **Fra il momento in cui è generato e il momento in cui è inviato al processo, il Segnale è Pendente** (*pending*).

Di norma, un Segnale Pendente è inviato a un processo appena il processo è scelto per l'esecuzione, *oppure* immediatamente se il processo è già in esecuzione.

A volte invece è necessario assicurare che un segmento di codice non sia interrotto dalla consegna di un segnale.

- In questo caso, possiamo aggiungere un segnale alla **"Maschera dei Segnali del Processo"**, cioè **un insieme di segnali la cui ricezione è attualmente bloccata**.
 - Varie system call permettono ai processi di aggiungere e rimuovere segnali dalla propria maschera dei segnali.

Al momento della ricezione di un segnale, un processo continua con una delle seguenti azioni di default, a seconda del segnale:

- Il segnale è ignorato; in questo caso viene scartato dal kernel e non ha effetti sul processo (il processo non è neppure informato del fatto che quel segnale è occorso).
- Il processo viene terminato (*killed*).
 - Questa terminazione è detta anche *abnormal process termination*, opposta alla terminazione normale del processo, che occorre quando un processo termina usando `exit()`
- Il processo viene bloccato (*stopped*): l'esecuzione è in questo caso sospesa.
- L'esecuzione del processo è ripresa (*resumed*) dopo essere stata bloccata in precedenza.
- È generato un file contenente un *core dump file*, e il processo viene terminato.
 - Un **File con Core Dump** contiene un'immagine della memoria virtuale del processo; tale immagine può essere caricata in un debugger per ispezionare lo stato del processo al momento della terminazione.

Classi di Segnali

- Le **Trap**: **segnali generati da eventi prodotti da un processo e inviati al processo stesso**.
 - Alcune trap sono causate da comportamenti errati del processo stesso, e immediatamente inviate al processo che normalmente reagisce terminando.
 - Per esempio: tentativi di divisione per zero (`SIGFPE`), indirizzamento errato degli array (`SIGSEGV`), tentativo di eseguire istruzioni privilegiate (`SIGILL`), etc..
 - Gli **Interrupt** sono **segnali inviati ad un processo da un agente esterno**: l'utente o un altro processo
 - Utente: CTRL-C (invia `SIGINT`), CTRL-Z (invia `SIGSTOP`), Comando `kill -s SIGNAL PID`
 - Altro processo: `kill(PID, SIGNAL)`
-

Vari tipi di Segnali

Vediamo ora una carrellata di tutti i principali segnali:

- **SIGABRT;** Un processo riceve questo segnale quando invoca la funzione `abort()`.
- **SIGALRM;** Segnale inviato dal kernel a un processo al momento del raggiungimento dello zero di un timer.
- **SIGCHLD;** Segnale inviato dal kernel a un processo genitore quando:
 - uno dei figli termina *Oppure*
 - uno dei suoi figli è bloccato o risvegliato da un segnale.
- **SIGCONT;** Due casi:
 - 1) Quando viene inviato a un processo bloccato (*stopped*), questo segnale **causa il risveglio del processo (*resume*)**, cioè che il processo venga schedato per successivamente essere eseguito.
 - 2) Quando è ricevuto da un processo che non è bloccato, **questo segnale è ignorato di default**.
 - Un processo può intercettare questo segnale, in modo da eseguire qualche azione particolare al momento della ripresa dell'esecuzione.
- **SIGINT;** Quando l'utente digita il carattere di Interrupt (Control-C), il terminale invia questo segnale al gruppo del processo stesso (...). **L'azione di default per questo segnale è terminare il processo.**
- **SIGKILL;** È il segnale sicuro di kill. **Non può essere bloccato, ignorato, o intercettato da un handler, e quindi termina sempre un processo.**
- **SIGPIPE;** Segnale generato quando un processo tenta di scrivere su un pipe o un FIFO per il quale non c'è un corrispondente processo lettore.
- **SIGSEGV;** Segnale generato quando un programma tenta un riferimento in memoria non valido.
Il riferimento può non essere valido perché la pagina riferita non esiste oppure il processo ha tentato di modificare una locazione in read-only memory, o il processo ha tentato di accedere a una parte della memoria del kernel durante l'esecuzione in user mode.
- **SIGSTOP** Segnale per il blocco (stop) sicuro. **Non può essere bloccato, ignorato, o intercettato da un handler, e quindi blocca sempre un processo.**
- **SIGTERM.** Segnale standard utilizzato per terminare un processo;
 - segnale inviato di default dai comandi `kill` e `killall`.
 - **È errato inviare** esplicitamente il segnale `SIGKILL` a un processo (usando `kill -KILL`).
 - **Un'applicazione ben progettata deve avere un handler per `SIGTERM` che consenta di pulire i file temporanei e di rilasciare le altre risorse.**
 - L'uccisione di un processo con `SIGKILL` bypassa l'handler di `SIGTERM`, e quindi **si dovrebbe sempre prima cercare di terminare un processo con `SIGTERM`, e tenere `SIGKILL` come ultima scelta per terminare i processi che non rispondono a `SIGTERM`.**
- **SIGTRAP;** Segnale utilizzato per implementare i breakpoint in fase di debugging e per la tracciatura delle system call.
- **SIGTSTP;** Quando l'utente digita il carattere di Sospensione (Control-Z), il terminale invia questo segnale al gruppo del processo stesso (...). **L'azione di default per questo segnale è sospendere il processo.**
- **SIGUSR1.** Questo segnale e **SIGUSR2** sono disponibili per fini specificati dal programmatore.
 - **Il kernel non genera mai questi segnali per un processo.**
 - I processi possono utilizzare questi segnali per notificarsi a vicenda eventi, o per sincronizzarsi

Extra: Systemcall per Inviare Segnali

La systemcall `kill()` : permette di inviare un segnale `sig` (specificato) ad altri processi

- L'argomento `pid` identifica 1+ processi a cui inviare il segnale; può essere interpretato in 4 modi:
 - `pid > 0`: il segnale è inviato al processo identificato da `pid`.
 - `pid == 0`: il segnale è inviato a tutti i processi nello stesso gruppo del chiamante.
 - `pid < -1`: il segnale è inviato a tutti i processi nel gruppo del processo il cui PID è uguale al valore assoluto di `pid`.
 - `pid == -1`: (broadcast signal) il segnale è inviato a tutti i processi per i quali il processo ha i permessi di inviare un segnale, eccetto `init` ed il chiamante.
- Se nessun processo corrisponde PID predefinito, `kill()` fallisce setta `errno` a `ESRCH`
- Se il processo esiste, ma non si hanno i permessi per inviare un segnale, allora `errno = EPERM`
- Se l'argomento `sig` è settato a 0 (detto *null signal*), non è inviato alcun segnale.
 - In questo caso `kill()` esegue unicamente un controllo degli errori per vedere se è possibile inviare segnali al processo:
 - Il *null signal* può essere utilizzato per testare se un processo con un certo PID esiste. Se la chiamata va a buon fine, sappiamo che il processo esiste.

```
#include <signal.h>

int kill(pid_t pid, int sig);

//Returns 0 on success, or -1 on error
```

al
e

La systemcall `raise()` :

La syscall `raise()` , permette di inviare un segnale `sig` (specificato) al processo stesso che invoca la systemcall

```
#include <signal.h>

int raise(int sig);

//Returns 0 on success, or nonzero on error
```

La systemcall `alarm()` :

La syscall `alarm()` , permette di inviare un segnale SIGALRM al processo stesso che invoca la syscall dopo un numero di secondi.

```
#include <unistd.h>

unsigned int alarm (unsigned int sec);
```

Valore di ritorno:

- 0 se non ci sono precedenti invocazioni di `alarm` pendenti
 - il numero di secondi che mancano alla scadenza dell'allarme precedente
-

Invece di accettare l'azione di default per un particolare segnale, un programma può modificare l'azione da intraprendere al momento della consegna del segnale.

Questo è noto come **Impostazione dell'Handler del Segnale**.

Un programma può impostare uno dei seguenti handler del segnale:

1. Azione di Default;

- Ad esempio, il segnale SIGTERM di solito richiede la terminazione di un processo. Quindi, se si imposta l'handler del segnale su "Azione di Default", il sistema operativo intraprenderà automaticamente l'azione predefinita associata a quel segnale quando viene ricevuto dal processo.

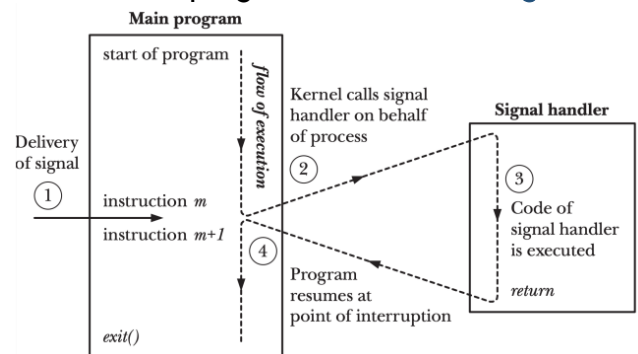
2. Il segnale è Ignorato;

3. Viene Eseguito un **Signal Handler**;

Signal Handler

Un **Signal Handler** (o “Gestore di Segnali”) è una funzione, scritta dal programmatore, che esegue azioni appropriate in risposta alla ricezione di uno specifico segnale.

- Quando un handler è invocato in risposta alla ricezione di un segnale, diciamo che il segnale è stato gestito (*handled*) o intercettato (*caught*).
- Quando l'Handler restituisce, l'esecuzione del programma riprende dal punto in cui l'Handler lo aveva interrotto.



Cambiare l'Handler dei Segnali

I sistemi UNIX forniscono due modi per cambiare l'handler di un segnale:

- `signal()` - **DEPRECATA! Non va usata!**
 - è l'API originale per assegnare l'handler di un signal, e fornisce un'interfaccia più semplice di `sigaction()`.
 - **ATTENZIONE:** vi sono differenze nel comportamento di `signal()` fra le varie implementazioni di UNIX. à da evitare!
- **`sigaction()`:**
 - Gestione dei segnali più completa e robusta rispetto all'uso di `signal()`
 - Parametri:
 - `signum`: segnale che deve essere gestito
 - `act`: nuovo handler per il segnale (se NULL, nessun cambiamento)
 - `oldact`: puntatore al vecchio handler

man 2 sigaction

```
#define _GNU_SOURCE
#include < signal .h >

int sigaction ( int signum , const struct sigaction * act, struct
sigaction * oldact ) ;
```

```
struct sigaction new, old;

// set new handler to new
sigaction(signum, new, NULL);

// current handler in old
sigaction(signum, NULL, old);

// do both
sigaction(signum, new, old);
```

Sigaction Structure

Principali elementi:

- `sa_handler`: puntatore alla funzione di handling
- `sa_mask`: maschera per indicare i segnali bloccati (macerati) durante l'esecuzione dell'handler
 - si pensi ad esempio all'esecuzione di una sezione critica
- `sa_flags`: maschera bitwise (or-style) che modifica il comportamento del gestore del segnale

```
#define _GNU_SOURCE
#include <signal.h>

struct sigaction {
    void (*sa_handler) (int signum) ;
    sigset_t sa_mask ;
    int sa_flags ;
    // plus others (for advanced users)
};
```

Problema: Handler definiti dall'Utente e Variabili Globali

- Le funzioni di gestione degli handler vengono ereditate dai processi figli dopo una `fork()`
- Le variabili globali definite nel programma sono visibili sia dalle funzioni handler che dal resto del programma
 - Questo può essere utile ...
 - **Ma.... cosa succede quando un handler modifica il valore di una variabile globale il cui valore non dovrebbe essere modificato?** Alcune system call utilizzano strutture dati globali e quindi il loro utilizzo all'interno di un handler può generare problemi
 - Esempio: `printf()`; Cosa succede se un segnale interrompe l'esecuzione di una `printf`, che poi viene anche utilizzata all'interno dell'handler?

Fortunatamente ci sono funzioni che sono definite come "AS-Safe" (Asynchronous Signal-Safe)

- Esempio: `write()`
 - **Consiglio:** ogni volta che si vuole utilizzare una system call in un handler, controllare se è AS-safe nella documentazione
- `errno` è una variabile globale e il suo valore potrebbe essere sovrascritto durante l'esecuzione di un handler; **Consiglio:** salvare e ripristinare il valore di `errno` nell'handler
-

Gestire più Segnali con un Unico Handler

Lo switch può essere utilizzato per discriminare il tipo di segnale che l'handler ottiene come argomento

```
void handle_signal(int signum) {
    /* signal signum triggered the handler */
    switch (signum) {
        case SIGINT:
            /* handle SIGINT */
            break;
        case SIGALRM:
            /* handle SIGALRM */
            break;
        /* other signals */
    }
}
```

Quando un segnale `signum` è consegnato ad un processo, durante l'esecuzione del suo handler, il segnale `signum` è bloccato...A meno che il flag `SA_NODEFER` è settato nella `sigaction()`.

```
struct sigaction sa;
sigset_t my_mask;

sa.sa_handler = handle_signal;
sa.sa_flags = SA_NODEFER; // allow nested invocations
sigemptyset(&my_mask); // do not mask any signal
sa.sa_mask = my_mask;
sigaction(SIGUSR1, &sa, NULL); // set the handler
```


Signal Mask

La **Signal Mask** maschera è la collezione di segnali attualmente bloccati

- Ogni processo ha la propria Maschera dei Segnali: un nuovo processo eredita la maschera del genitore
- Le maschere sono di tipo `sigset_t`
- Le funzioni per la manipolazione dei set sono:

```
int sigemptyset(sigset_t *set);
int sigfillset(sigset_t *set);
int sigaddset(sigset_t *set, int signum);
int sigdelset(sigset_t *set, int signum);
int sigismember(const sigset_t *set, int signum);
```

Impostare la Maschera dei Segnali di un Processo:

Per impostare la maschera di segnali bloccati durante l'esecuzione dei processi si usa la syscall `sigprocmask()`:

- L'argomento `how` può assumere i seguenti valori:
 - `SIG_BLOCK`: i segnali nel set sono aggiunti a quelli bloccati;
 - `SIG_UNBLOCK`: i segnali nel set sono rimossi dalla maschera esistente;
 - `SIG_SETMASK`: il set diventa la nuova maschera del segnale.
- L'argomento `oldset` è la vecchia maschera

```
#include <signal.h>
int sigprocmask(int how,
                const sigset_t *set,
                sigset_t *oldset);
Returns 0 if successful; otherwise the value -1 is returned and the
global variable errno is set.
```

- L'Handler può comunque essere interrotto da altri segnali
 - o dallo stesso segnale, nel caso `SA_NODEFER` sia settato.
- Quando l'Handler termina, la Signal Mask (insieme di segnali bloccati) è reimpostata al suo valore precedente la sua esecuzione, indipendentemente dalle possibili manipolazioni dei segnali bloccati eventualmente presenti nell'Handler.
- Se un segnale viene generato mentre un altro stesso segnale è in stato pendente, il nuovo segnale e quello pendente sono accorpati in uno; si parla di "*Merged Signal*".
- Un gestore di segnali non può essere utilizzato per contare il numero di segnali ricevuti

```
struct sigaction sa;
sigset_t my_mask;

sa.sa_handler = &handle_signal; // handler
sa.sa_flags = 0; // No special behaviour

// Create an empty mask
sigemptyset(&my_mask); // Do not mask any signal
sigaddset(&my_mask, signal_to_mask_in_handler);
sa.sa_mask = my_mask;

sigaction(SIGINT, &sa, NULL); // Set the handler
```


ATTENZIONE: Invio di Segnali a Processi Sospesi

Cosa succede quando un processo è in uno stato di waiting e arriva un segnale? **Dipende**

Un processo è in uno stato di waiting quando è deciso dallo scheduler o quando ha eseguito una syscall bloccante (come ad esempio `wait()`, `pause()` o `sleep()`).

Concentriamoci su quest'ultimo caso.

Quando un processo è in attesa o sospeso (stato di waiting):

1. Il processo non è in esecuzione (è sospeso)
2. La funzione dell'Handler è eseguita normalmente
3. Quando l'Handler ritorna, ci sono due casi:
 - A. **La syscall restituisce un errore, con errno settato a EINTR;**
 - Per capire questo concetto consideriamo un esempio: un processo Padre vuole aspettare la terminazione di 5 figli. Come mi accorgo che i 5 figli sono terminati? Con la `wait()`; quando restituisce -1 e `errno=ECHILD`.
 - Invece: se la `wait()` è bloccata e arriva un segnale, accade che restituisce -1 e `errno=EINTR`.
 - Quindi è bene controllare `errno` per sapere quale dei due eventi si è verificato
 - **ATTENZIONE:** Se il flag `SA_RESTART` è settato in `sa_flags` (nella syscall `sigaction()`) l'esecuzione della syscall riprende dopo l'esecuzione dell'handler

B. **La syscall viene automaticamente ripresa.**

A o B? **Dipende dal sistema operativo;**

- Nella sezione «Interruption of System Calls ... by signal handlers» («man 7 signal») ci sono le indicazioni puntuali

Extra: è bene sapere (ma non approfondiremo...) che è possibile anche la gestione sincrona dei segnali: un processo attende la ricezione di un particolare segnale (...)